Reasoning and Inference in Intelligent Systems

Lecturer: Jeewaka Perera
Module Code: SE3062
Faculty of Computing, SLIIT

Content

1. Propositional Logic
2. Predicate / First-Order Logic
3. Probabilistic Logic

Why Logic in AI?

- **Represent knowledge in a structured way**
  - Move from raw data (numbers, text) to structured facts and rules.
  - Example: Instead of storing *"Rain=Yes"*, logic allows *"If Rain then WetGround."*
- **Infer new facts from existing facts**
  - Enables reasoning: deducing unseen information from what is known.
  - Example: If *"All humans are mortal"* and *"Socrates is human"*, then → *"Socrates is mortal."*
- **Make decisions under certainty and uncertainty**
  - Certainty: Classical logic (True/False).
  - Uncertainty: Probabilistic reasoning (likelihoods, risks).
  - Example: Medical AI suggesting diagnosis with confidence scores.

Considerations (Why it matters):

- Without logic, AI is limited to **pattern recognition only** (e.g., ML black-boxes).
- Logic introduces **transparency**: each step in reasoning is explainable and auditable.
- Essential for **critical domains** (law, healthcare, finance) where justifications matter.
- 💡 **Tip for Students:**
  When designing AI, always ask: *"Can I explain how the system reached this decision?"* Logic helps make this possible.

Discussion

Imagine a medical AI gives a diagnosis: would you trust it more if it explains *why* step by step, or just gives a probability?

Can you think of an example where pattern-based AI (like ML) failed because it lacked reasoning?

In what situations might probabilistic logic be more useful than strict true/false reasoning?

Discussion

1. Trust in Medical AI: Explanation vs. Probability

- **Scenario A (Black Box):**
  - A medical AI says: *"Patient has lung cancer with 78% probability."*
  - No reasoning given → doctors may hesitate to trust it.
- **Scenario B (Explainable):**
  - AI says: *"Patient has lung cancer (78% confidence) because:
    - X-ray shows irregular mass in left lung.
    - Patient history includes heavy smoking.
    - Cough with blood present."*
  - Each step links to a **logical rule** (If mass ∧ smoker ∧ cough-blood ⇒ high likelihood of cancer).
  - Doctors are more likely to **trust and act** on this system.
- 💡 **Takeaway:** In **safety-critical systems**, explanation matters as much as accuracy.

Discussion

2. Failures of Pattern-Based AI (No Reasoning)

- **Example: Amazon's Hiring Algorithm (2018)**
  - The system learned from past resumes → developed **gender bias** (downgrading women's resumes).
  - Why? It recognized statistical patterns without **understanding rules of fairness**.
- **Example: Tesla Autopilot Incident**
  - Vision-based system failed to recognize a white truck against a bright sky → tragic accident.
  - A logic-based rule like *"All vehicles must be detected, regardless of brightness/contrast"* could have provided redundancy.
- 💡 **Takeaway:** Pure ML can overfit to data patterns. Logic adds **domain knowledge constraints** to prevent obvious failures.

Discussion

3. When Probabilistic Logic > Strict Logic

- **Medical Diagnosis:**
  - Patient has fever, cough, fatigue.
  - Strict logic: *If cough ∧ fever ⇒ Flu.*
  - But symptoms could also mean *COVID, dengue, pneumonia*.
  - Probabilistic logic assigns weights:
    - Flu = 0.6, Dengue = 0.25, COVID = 0.15.
- **Spam Filtering:**
  - Email contains words: "Free," "Win," "Offer."
  - Strict rule: If contains "Free" ⇒ Spam → ❌ will misclassify normal emails ("Free Seminar on AI").
  - Probabilistic: P(Spam|words) = 0.85 → more robust.
- **Weather Forecasting:**
  - Logic: *If clouds ⇒ rain.*
  - Reality: Clouds sometimes produce no rain.
  - Probabilistic: P(Rain|Clouds) = 0.7, P(NoRain|Clouds) = 0.3.
- 💡 **Takeaway:** Probabilistic logic is essential in **uncertain, noisy, or overlapping domains**.

What is Propositional Logic?

- **Definition**
  - Propositional logic (also called *sentential logic*) is a formal system where **propositions** (statements) can be assigned one of two truth values: **True (T)** or **False (F)**.
  - Propositions are treated as indivisible units – we don't look inside their meaning, only at whether they are true or false.

- **Properties**
  - **Atomic Propositions:** Simple statements that cannot be broken down further.
    - Example: "It is raining."
  - **Compound Propositions:** Built from atomic ones using logical connectives (∧, ∨, ¬, ⇒, ⇔).
    - Example: "It is raining AND it is cold."

More on Propositional Logic

- **Valid Propositions**
  - "It is raining." (True/False depending on weather)
  - "2 + 2 = 4." (Always True)
  - "Colombo is the capital of Sri Lanka." (True/False depending on fact-checking)
- **Non-Propositions (Invalid Cases)**
  - "Read this!" (Command → not True/False)
  - "Who are you?" (Question → not True/False)
  - "x + 2 = 5" (Expression with variable x → not a proposition until x is defined)
- **Common Misconceptions**
  - "Maybe" or "Sometimes" are not valid truth values in propositional logic.
  - A proposition must always have a definite True/False value at a given time.
- **Considerations & Tips**
  - When you see a statement, ask: *"Can I clearly say it's True or False?"*
    - If yes → it's a proposition.
    - If not → it's not propositional logic.
  - In computer science, propositions often represent conditions in programs (e.g., "x > 0").

Discussion

- Which of these are propositions? Why/why not?
  - “The moon is made of cheese.”
  - “Close the door.”
  - “5 is an odd number.”
  - “x + y = 10.”

Syntax of Propositional Logic

- **Propositional Symbols**
  - Represent entire statements.
  - Usually uppercase letters: P, Q, R, …
  - Example:
    - P: “It is raining.”
    - Q: “The ground is wet.”

Logical Connectives

- **Negation (¬P)** – *"NOT P"*
  - True if P is False; False if P is True.
  - Example: P = "It is raining." → ¬P = "It is not raining."
- **Conjunction (P ∧ Q)** – *"P AND Q"*
  - True only if both P and Q are True.
  - Example: "It is raining AND it is cold."
- **Disjunction (P ∨ Q)** – *"P OR Q"*
  - True if at least one of P, Q is True.
  - Example: "It is raining OR it is cold."
- **Implication (P ⇒ Q)** – *"IF P THEN Q"*
  - False only when P is True and Q is False.
  - Example: "If it rains, then the ground is wet."
- **Biconditional (P ⇔ Q)** – *"P IF AND ONLY IF Q"*
  - True if both P and Q have the same truth value.
  - Example: "You can graduate ⇔ You pass all modules."

Summary

Connective	True When	False When
$\neg P$	P = F	P = T
$P \wedge Q$	P = T, Q = T	Any other case
$P \vee Q$	At least one True	Both False
$P \Rightarrow Q$	All except (P = T, Q = F)	P = T, Q = F
$P \Leftrightarrow Q$	Both equal	Different values

Discussion

- Translate into propositional logic:
  - “If I study, I will pass the exam.”
  - “It is cold or it is snowing.”
  - “I will go out if and only if it is Saturday.”

Strengths & Limitations of Propositional Logic

- **Strengths**
  - **Simple, clear, and easy to compute**
    - Computers can quickly evaluate truth values using truth tables.
    - Example: In digital circuits, logic gates (AND, OR, NOT) directly correspond to propositional operators.
  - **Allows combining statements using connectives** (negation, conjunction, disjunction, implication, equivalence).
    - Example:
      - P = "It is raining."
      - Q = "The ground is wet."
      - P ⇒ Q: If it rains, the ground is wet.
  - **Well-defined semantics**
    - Each connective has an unambiguous truth table definition.
      sent "likely," "probable," or degrees of belief.

- **Limitations**
  - **Cannot represent relationships between objects**
    - Example: Cannot represent "John is the brother of Mary." It only treats whole statements as indivisible symbols.
  - **No generalization**
    - Cannot express "All humans are mortal" or "Some cats are black."
    - Instead, every fact must be written separately: Mortal(John), Mortal(Mary), Mortal(Alice), etc.
  - **Scalability issues**
    - As the domain grows, the number of separate propositions becomes unmanageable.
    - Example: Representing 100 students requires 100 individual propositions.
  - **No handling of uncertainty**
    - Every statement must be either True or False.
    - Cannot represent "likely," "probable," or degrees of belief.

Considerations and Applications

- **Considerations and Applications**
- Well-suited for:
  - Digital circuits (logic gates).
  - Simple control systems (e.g., alarms, switches, rule triggers).
- Limited for:
  - Complex AI tasks requiring reasoning about people, relationships, or uncertainty.
  - Large knowledge bases where generalization is required.

Discussion

- Suppose we design a burglar alarm system:
  - P = “Window is open.”
  - Q = “Motion detected.”
  - R = “Alarm rings.”

Represent this in propositional logic.

Why would propositional logic fail if we needed to represent who opened the window (e.g., Alice vs. Bob)?

FIRST ORDER LOGIC

Problem with Propositional Logic??

- **Why Propositional Logic Fails for "Who" Opened the Window:**
- Propositional logic treats each fact as an indivisible whole.
- We would need **separate propositions for each case**:
  - P1 = "Alice opened the window."
  - P2 = "Bob opened the window."
- As the number of possible intruders grows, the number of propositions explodes.
- There is no way to capture the **relationship**: *Opened(Alice, Window)*.

How First-Order Logic (FOL) Fixes This

- FOL lets us talk about **objects** and their **relations**.
- Examples of facts:
  - Opened(Alice, Window1)
  - Opened(Bob, Window2)
- General rule in FOL:
  - $\forall x$ (Opened(x, Window) $\wedge$ MotionDetected $\Rightarrow$ AlarmRings)
- **Why This Matters**
  - Instead of writing **separate propositions** for every possible case (Alice, Bob, etc.), FOL allows **generalization**.
  - This is the next step: moving from simple true/false statements → to a system that can model the **structure of the world**.

What is FOL?

- **First-Order Logic (Predicate Logic)** extends propositional logic by introducing:
  - **Objects** – individual entities (Alice, Bob, Window1).
  - **Predicates** – properties or relationships of objects (Opened(x, y), Student(x)).
  - **Quantifiers** – to generalize statements:
    - ∀ (for all)
    - ∃ (there exists)
- Why FOL is Needed
- Propositional logic is limited to simple True/False statements.
- FOL allows us to:
  - Represent relationships between objects.
  - Express general statements like “All humans are mortal.
  - ”Handle more complex knowledge bases in AI.
- Propositional: “John is a student” → just one atomic symbol (S).
- FOL: Student(John) – now we can attach meaning and extend it.

Syntax of First-Order Logic (FOL)

1. Constants

- Represent **specific objects** in the domain.
- Examples: John, 3, SLIIT.
- Meaning: constants always point to a particular entity.
- Analogy: proper nouns in English.

2. Variables

- Represent **arbitrary elements** from the domain.
- Examples: x, y, z.
- Used with quantifiers to make general statements.
- Example: "∀x Student(x)" → for *every x*, x is a student.

3. Predicates

- Express **properties** of objects or **relations** between objects.
- Example:
  - Student(x) → x is a student.
  - Brother(x,y) → x is the brother of y.
- Predicates give **structure** to the world (unlike propositional symbols).

4. Functions

- Map an object to another object in the domain.
- Example:
  - Father(x) → returns the father of x.
  - Square(x) → returns the square of x.
- Functions are different from predicates:
  - Predicate(Student(x)) → true/false.
  - Function(Father(John)) → returns an object (e.g., Peter).

5. Connectives

Logical operators to build complex sentences:

- ∧ = AND
- ∨ = OR
- ¬ = NOT
- ⇒ = IF…THEN
- ⇔ = IF AND ONLY IF

6. Quantifiers

Special symbols to make **generalized statements**:

- ∀ = Universal quantifier → "For all x …"
  - Example: ∀x Human(x) ⇒ Mortal(x).
- ∃ = Existential quantifier → "There exists at least one x …"
  - Example: ∃x Student(x) ∧ Smart(x).

Atomic vs Complex Sentences

Atomic Sentences

- **Definition:** Simplest statements in FOL. Cannot be broken down further.
- **Form:** Predicate + constants/variables.
- **Examples:**
  - Student(John) → John is a student.
  - Taller(Alice, Bob) → Alice is taller than Bob.
- **Truth value:** Determined directly from the interpretation (the model).

Complex Sentences

- **Definition:** Built by combining atomic sentences using logical connectives (¬, ∧, ∨, ⇒, ⇔) and quantifiers (∀, ∃).
- **Examples:**
  - ∀x Student(x) ⇒ Human(x) → All students are humans.
  - ¬Student(Alice) → Alice is not a student.
  - Student(John) ∧ Smart(John) → John is a student and smart.

Discussion

- How do we represent the fact that *brothers are siblings* in FOL?
- How do we capture the idea that *sibling is symmetric* (if x is sibling of y, then y is sibling of x)?
- How can we formally say that *a mother is a female parent*?
- Translate into FOL:
  - "All cats are animals."
  - "Some students like AI."

Inference in FOL

- **Universal Instantiation (UI)**
  - **Definition:** From a universally quantified statement, we can infer it for any specific constant.
  - **Rule:** $\forall x\ P(x) \Rightarrow P(c)$, where *c* is any constant in the domain.
  - **Example:**
    - $\forall x\ King(x) \Rightarrow Mortal(x)$
    - King(John)
    - Therefore Mortal(John)
  - **Explanation:** If the rule applies to *all x*, then it must apply to John too.

- **Existential Instantiation (EI)**
  - **Definition:** From an existential statement, we can introduce a new constant to represent the object that exists.
  - **Rule:** $\exists x\ P(x) \Rightarrow P(c1)$, where c1 is a new constant (not used before).
  - **Example:**
    - $\exists x\ Student(x)$
    - Introduce new constant C1 → Student(C1).
  - **Explanation:** If some student exists, we can create a placeholder constant (C1) to reason about that student.

Inference in FOL…

Examples

- ∀x Human(x) ⇒ Mortal(x)
  - Human(Socrates)
  - ⇒ Mortal(Socrates)
- ∃x Animal(x) ∧ Mammal(x)
  - Introduce constant A1 → Animal(A1) ∧ Mammal(A1).

- **Common Pitfalls**
  - **UI mistake:** Forgetting to substitute the variable with the constant.
  - **EI mistake:** Reusing the same constant for different existential statements (always introduce a fresh one).

- **Why Important?**
  - These inference rules are the **building blocks of automated reasoning systems**.
  - Used in **forward chaining, backward chaining, and Prolog-style inference.**

Generalized Modus Ponens (GMP)

Definition

- An **inference rule in First-Order Logic (FOL)**.
- Generalizes propositional *modus ponens* to handle **variables and predicates**.
- Form:
  - If $(p_1 \wedge p_2 \wedge \cdots \wedge p_n \Rightarrow q)$and all $p_i$are true (after substitution),
  - Then $q$is true.

Steps in GMP

1. Take a universally quantified rule (e.g., ∀x).
2. Match (unify) the premises with facts in the knowledge base.
3. Apply the substitution to the conclusion.
4. Add the conclusion as a new fact.

Example

- **Example 1 (Simple)**
  - Rule: $\forall x\ [King(x) \land Greedy(x) \Rightarrow Evil(x)]$
  - Facts: King(John), Greedy(John)
  - Substitution: {x/John}
  - Inference: Evil(John)
- **Example 2 (Chain reasoning)**
  - Rule: $\forall x,y,z\ [Parent(x,y) \land Parent(y,z) \Rightarrow Grandparent(x,z)]$
  - Facts: Parent(Ada,Ben), Parent(Ben,Chloe)
  - Substitution: {x/Ada, y/Ben, z/Chloe}
  - Inference: Grandparent(Ada,Chloe)

Forward vs Backward Chaining

Forward Chaining (Data-driven reasoning)

- **Definition:** Start with known facts, repeatedly apply rules, and derive all possible new facts until the goal is reached.
- **Process:**
  - Begin with facts in the Knowledge Base (KB).
  - Match facts with rule premises.
  - Infer new facts and add them to the KB.
  - Continue until no new facts can be added.
- **Use case:** Useful when many facts are known, and you want to derive all possible conclusions.
- **Analogy:** Like a doctor running **tests → symptoms → possible diagnoses**.

Forward vs Backward Chaining

Backward Chaining (Goal-driven reasoning)

- **Definition:** Start with a query (goal), work backwards through rules to check if it can be satisfied by known facts.
- **Process:**
  - Start with the query.
  - Find rules that could conclude it.
  - Reduce query to sub-queries (premises of rules).
  - Continue until you reach facts in KB.
- **Use case:** Efficient when the query is specific and KB is large.
- **Analogy:** Like a lawyer starting from a **claim → checking laws → finding supporting evidence**.

Example

- KB:
  - $\forall x$ Student(x) $\Rightarrow$ Human(x)
  - Student(Alice)

- ***Query: Human(Alice)?***

- **Forward Chaining:**
  - From Student(Alice), apply rule → Human(Alice).
- **Backward Chaining:**
  - Start with Human(Alice).
  - Rule says: if Student(Alice), then Human(Alice).
  - Check KB → Student(Alice) is true → therefore Human(Alice).

PROBABILISTIC LOGIC

Why Probabilistic Logic?

- Information is often **uncertain, incomplete, noisy**.
- Medical example:
  - FOL: “If Cough(x) ⇒ Flu(x).”
  - Reality: Cough may mean **Flu, Cold, or Allergy**.
- Sensor example:
  - FOL: “If MotionDetected ⇒ Intruder.”
  - Reality: Could be **intruder, pet, or false alarm**.

The Need for Probabilistic Logic

- Pure logic can’t capture **likelihoods or confidence**.
- Probabilistic logic = **Logic + Probability**.
  - Lets us assign degrees of belief:
    - P(Flu | Cough) = 0.6
    - P(Cold | Cough) = 0.3
    - P(Allergy | Cough) = 0.1
- Logic tells us what follows if something is True.
- Probability tells us how likely something is.
- Together → more realistic reasoning for AI.

Bayes' Theorem

- **What is Bayes' Theorem?**
  - A way to **update our belief** about a hypothesis when we get new evidence.
  - Connects **prior knowledge** with **new information**.

- **The Formula**

$$P(H \mid E) = \frac{P(E \mid H) \cdot P(H)}{P(E)}$$

- **P(H|E): Posterior probability** → probability of hypothesis Hgiven evidence E.
- **P(E|H): Likelihood** → probability of evidence Eoccurring if $H$is true.
- **P(H): Prior probability** → our initial belief about H, before seeing evidence.
- **P(E): Marginal probability** → probability of evidence under all possible hypotheses.

Where Bayes' Theorem Fits In

- Bayes' theorem gives the **mathematical foundation** for updating probabilities when new evidence arrives.
- Probabilistic logic needs a way to **revise beliefs** dynamically.
- Without Bayes, probabilities would be static; with Bayes, they adapt to evidence.
- Example:
  - Probabilistic rule: "Cough suggests Flu with probability 0.7."
  - Bayes lets us **combine prior belief** (flu is rare = 5%) with **new evidence** (cough) to compute a posterior belief (flu likelihood jumps to 30%).

Step-by-Step Example

- Start with a **prior belief** (P(H)).
  - Example: 5% of people have flu.
- Gather **evidence** (E).
  - Example: a person is coughing.
- Check how likely the evidence is **if hypothesis is true** (P(E|H)).
  - Example: 80% of flu patients cough.
- Adjust your belief using Bayes → get a **posterior probability**.

Example

- Prior: P(Flu) = 0.05
- Likelihood: P(Cough|Flu) = 0.8
- Noise: P(Cough|¬Flu) = 0.1
- Marginal: P(Cough) = (0.8×0.05) + (0.1×0.95) = 0.135
- Posterior:

$$P(Flu \mid Cough) = \frac{0.8 \times 0.05}{0.135} \approx 0.296$$

- So, if a patient coughs, the chance of flu rises from 5% (prior) → 30% (posterior).

Applications of Probabilistic Logic

- **Medical AI:** Use probabilistic logic for "symptom → disease" rules; use Bayes to update when multiple symptoms/evidence accumulate.
- **Search Engines / Information Retrieval:**
  - Query relevance can be seen as a hypothesis.
  - Bayes helps compute P(Document|Query).
  - This underlies **ranking algorithms** in search (retrieval models like probabilistic relevance).
- **Natural Language Processing:**
  - Probabilistic grammar rules + Bayes → parsing uncertain language input.
- **Robotics / Agents:**
  - Logic: "If obstacle detected ⇒ stop."
  - Probabilistic logic: "If sensor detects obstacle, 90% chance obstacle is real."
  - Bayes updates belief by combining multiple noisy sensor readings.

QUESTIONS